



インセプション

概要:

この文書はシステム管理に関連する演習です。

バージョン: 5.0

目次

I	前文	2
II	はじめに	3
III	一般的なガイドライン	4
IV	AI指示	5
V	必須項目	7
VI	Readme 要件	12
VII	検証の前提条件	13
VIII	ボーナスパート	14
IX	提出とピア評価	15

第I章 前文



第II章 導入

このプロジェクトは、Dockerを使用してシステム管理に関する知識を広げることを目的としています。新しい個人用仮想マシンで複数のDockerイメージを作成し、仮想化します。

第III章 一般的なガイ

ドライン

- このプロジェクトは仮想マシン上で実施する必要があります。
- プロジェクトの設定に必要なすべてのファイルは、`srcs`フォルダに配置してください。
- Makefileも必要であり、ディレクトリのルートに配置する必要があります。アプリケーション全体を設定しなければなりません（つまり、`docker-compose.yml`を使用してDockerイメージをビルドする必要があります）。
- この課題では、バックグラウンドによってはまだ習得していない概念を実践する必要があります。そのため、Dockerの使用に関するドキュメントや、この課題の完了に役立つと思われるあらゆる資料を積極的に参照することをお勧めします。

第4章

AI 指示

● 背景

学習過程において、AIは様々なタスクを支援できます。AIツールの多様な機能と、それらがどのように作業を支えられるかを探求する時間を取ってください。ただし、常に慎重に扱い、結果を批判的に評価することが重要です。コード、ドキュメント、アイデア、技術的な説明など、質問が適切に構成されていたか、生成された内容が正確であるかを完全に確信することはできません。仲間は、間違いや盲点を避けるための貴重なリソースです。

● 主なメッセージ

- ➡ 反復的・煩雑なタスクの削減にAIを活用する
- ➡ 将来のキャリアに役立つプロンプティングスキル（コーディング・非コーディング双方）を磨く。
- ➡ AIシステムの仕組みを理解し、一般的なリスク・バイアス・倫理的問題を予測・回避する能力を高めましょう。
- ➡ 同僚と協働することで、技術スキルとパワースキルの両方を継続的に構築する。
- ➡ 完全に理解し、責任を持てるAI生成コンテンツのみを使用してください。

● 学習者ルール：

- AIツールを十分に理解し、その仕組みを把握する時間を確保してください。これにより倫理的に使用でき、潜在的なバイアスを減らせます。
- プロンプトを入力する前に問題を振り返るべきです。これにより、正確な語彙を用いてより明確で詳細かつ関連性の高いプロンプトを作成できます。
- AIが生成した内容は、体系的に確認・検証・疑問視・テストする習慣を身につけるべきです。
- 常にピアレビューを求めること。自己検証のみに依存しないでください。

● フェーズ成果：

- 汎用的なプロンプティングスキルと特定分野向けプロンプティングスキルの両方を開発する。
- AIツールの効果的な活用で生産性を向上させる。
- 計算思考、問題解決能力、適応力、協働能力を継続的に強化する。

● コメントと例：

- 試験や評価など、真の理解を示す必要がある状況に頻繁に直面します。技術的スキルと対人スキルの両方を継続的に磨き、準備を整えましょう。
- 推論の解説や仲間との議論は、理解の不足点を明らかにすることが多い。ピアラーニングを優先事項とせよ。
- AIツールは特定の文脈を理解できず、汎用的な回答を提示しがちです。同じ環境にいる仲間は、より関連性が高く正確な洞察を提供できます。
- AIが最も可能性の高い答えを生成する傾向にあるのに対し、仲間は代替的な視点や貴重なニュアンスを提供できます。彼らを品質チェックポイントとして頼りましょう。

✓ 良い実践例：

AIに「ソート関数のテスト方法は？」と尋ねると、いくつかのアイデアが返ってくる。試してみて結果を同僚とレビューし、一緒にアプローチを洗練させる。

✗ 悪い実践例：

AIに丸ごと関数を書かせ、自分のプロジェクトにコピペする。ピア評価の際、その関数が何をするのか、なぜそうするのか説明できない。信頼性を失い——プロジェクトに失敗する。

✓ 良い実践例：

AIでパーサーの設計を支援してもらう。その後、論理をピアと確認しながら進める。2つのバグを発見し、一緒に書き直す——より良く、よりクリーンで、完全に理解できる形に。

✗ 悪い実践例：

プロジェクトの重要な部分のコードをCopilotに生成させた。コンパイルは通るが、パイプの処理方法を説明できない。評価時に正当化できず、プロジェクトは不合格となった。

第V章 必須部分

このプロジェクトでは、特定のルールに基づいて異なるサービスで構成される小規模なインフラストラクチャを構築します。プロジェクト全体は仮想マシン上で実施する必要があります。docker composeを使用してください。

各Dockerイメージは対応するサービスと同じ名前でなければならない。

各サービスは専用のコンテナで実行する必要があります。

パフォーマンス上の理由から、コンテナはAlpineまたはDebianの安定版から2つ前のバージョンで構築する必要があります。選択は自由です。

各サービスごとに独自のDockerfileを作成する必要があります。これらのDockerfileは、Makefileによってdocker-compose.yml内で呼び出される必要があります。

これは、プロジェクトのDockerイメージを自身でビルドする必要があることを意味します。したがって、既製のDockerイメージのプルや、DockerHubなどのサービス（Alpine/Debianはこの規則から除外されます）の使用は禁止されています。

次に、以下の設定が必要です：

- TLSv1.2 または TLSv1.3 のみをサポートする NGINX を搭載した Docker コンテナ。
- WordPress + php-fpm（インストールおよび設定済みであること）のみを含むDockerコンテナ（nginxなし）。
- nginxを含まないMariaDBのみのDockerコンテナ。
- WordPressデータベースを格納するボリューム。
- WordPressのウェブサイトファイルを含む2つ目のボリューム。
- コンテナ間の接続を確立するDockerネットワーク。

コンテナはクラッシュ時に再起動する必要があります。



Dockerコンテナは仮想マシンではありません。したがって、実行時に'tail -f'などのハック的な手法に基づくパッチの使用は推奨されません。デーモンの動作原理と、その使用が適切かどうかについて理解してください。



当然ながら、`network: host` や `--link`、`links:` の使用は禁止されています。ネットワーク設定行は必ず `docker-compose.yml` ファイルに記述してください。コンテナは無限ループを実行するコマンドで起動してはなりません。これはエントリポイントとして使用されるコマンド、またはエントリポイントスクリプト内で使用されるコマンドにも適用されます。以下に禁止されているハック的な手法の例を示します：`tail -f`、`bash`、`sleep infinity`、`while true`。

インセプション



PID 1 と Dockerfile 作成のベストプラクティスについて確認してください。

- WordPressデータベースには、管理者ユーザーを含む2人のユーザーが存在する必要があります。管理者のユーザー名には「admin/Admin」または「admin-istrator/Administrator」を含むことはできません（例：admin、administrator、Administrator、admin-123など）。



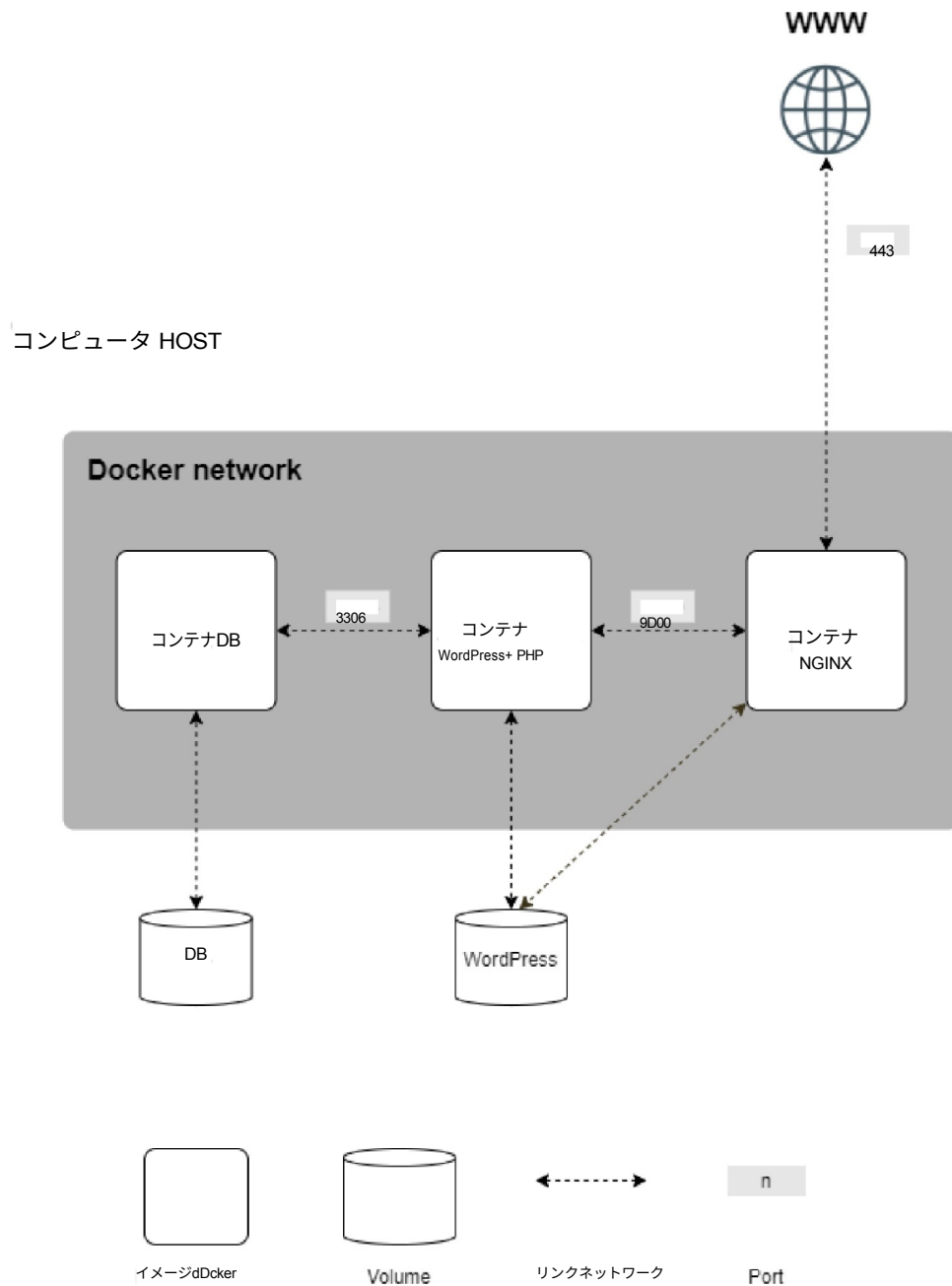
ボリュームは、Dockerを使用するホストマシンの`/home/login/data`フォルダで利用可能になります。もちろん、loginはご自身のログイン名に置き換えてください。

設定を簡素化するため、ドメイン名がローカルIPアドレスを指すように設定する必要があります。このドメイン名は `login.42.fr` でなければなりません。繰り返しになりますが、ご自身のログイン名を使用してください。たとえば、ログイン名が `wil` の場合、`wil.42.fr` は `wil` のウェブサイトを目指す IP アドレスにリダイレクトされます。



最新タグの使用は禁止されています。
Dockerfile内にパスワードを記述してはなりません。環境変数の使用が必須です。
また、環境変数の保存には `.env` ファイルの使用が必須です。機密情報の保存には Docker シークレットの使用を強く推奨します。Git リポジトリ内に保存された認証情報、API キー、パスワードはプロジェクト不合格となります。
NGINXコンテナは、TLSv1.2またはTLSv1.3プロトコルを使用し、ポート443経由でのみインフラストラクチャへの唯一のエントリポイントである必要があります。

期待される結果の例を示す図は以下の通りです：



開始

以下は想定されるディレクトリ構造の例です:

```
$> ls -alR total XX
drwxrwxr-x 3 wil wil 4096 42年4月20日 20:42 .
drwxrwxrwt 17 wil wil 4096 avril 42 20:42 ..
-rw-rw-r-- 1 wil wil XXXX 42年4月 20:42 Makefile
drwxrwxr-x 3 wil wil 4096 avril 42 20:42 secrets
drwxrwxr-x 3 wil wil 4096 avril 42 20:42 srcs

./secrets:
合計 XX
drwxrwxr-x 2 wil wil 4096 avril 42 20:42 .
drwxrwxr-x 6 wil wil 4096 avril 42 20:42 ..
-rw-r--r-- 1 wil wil XXXX 4月 42 20:42 credentials.txt
-rw-r--r-- 1 wil wil XXXX avril 42 20:42 db_password.txt
-rw-r--r-- 1 wil wil XXXX avril 42 20:42 db_root_password.txt

./srcs:
合計 XX
drwxrwxr-x 3 wil wil 4096 avril 42 20:42 .
drwxrwxr-x 3 wil wil 4096 avril 42 20:42 ..
-rw-rw-r-- 1 wil wil XXXX avril 42 20:42 docker-compose.yml
-rw-rw-r-- 1 wil wil XXXX avril 42 20:42 .env
drwxrwxr-x 5 wil wil 4096 avril 42 20:42 requirements

./srcs/requirements:
合計 XX
drwxrwxr-x 5 wil wil 4096 avril 42 20:42 .
drwxrwxr-x 3 wil wil 4096 avril 42 20:42 ..
drwxrwxr-x 4 wil wil 4096 avril 42 20:42 bonus
drwxrwxr-x 4 wil wil 4096 avril 42 20:42 mariadb
drwxrwxr-x 4 wil wil 4096 avril 42 20:42 nginx
drwxrwxr-x 4 wil wil 4096 avril 42 20:42 ツール
drwxrwxr-x 4 wil wil 4096 avril 42 20:42 wordpress

./srcs/requirements/mariadb:
合計 XX
drwxrwxr-x 4 wil wil 4096 avril 42 20:45 .
drwxrwxr-x 5 wil wil 4096 avril 42 20:42 ..
drwxrwxr-x 2 wil wil 4096 42年4月 20:42 conf
-rw-rw-r-- 1 wil wil XXXX 42年4月 20:42 Dockerfile
-rw-rw-r-- 1 wil wil XXXX avril 42 20:42 .dockerignore
drwxrwxr-x 2 wil wil 4096 4月 42日 20:42 tools [...]

./srcs/requirements/nginx:
合計 XX
drwxrwxr-x 4 wil wil 4096 avril 42 20:42 .
drwxrwxr-x 5 wil wil 4096 42年4月 20:42 ..
drwxrwxr-x 2 wil wil 4096 42年4月 20:42 conf
-rw-rw-r-- 1 wil wil XXXX 4月 42 20:42 Dockerfile
-rw-rw-r-- 1 wil wil XXXX avril 42 20:42 .dockerignore
drwxrwxr-x 2 wil wil 4096 avril 42 20:42 tools [...]

$> cat srcs/.env
DOMAIN_NAME=wil.42.fr # MYSQL
SETUP
MYSQL_USER=XXXXXXXXXXXX [...]
$>
```



明らかなセキュリティ上の理由から、あらゆる認証情報、APIキー、パスワードなどは、様々な方法/ファイルでローカルに保存し、gitで無視する必要があります。認証情報を公開状態で保存すると、プロジェクトの失敗に直結します。



変数（ドメイン名として）は、.envのような環境変数ファイルに保存できます

第6章

Readme 要件

GitリポジトリのルートディレクトリにはREADME.mdファイルを必ず配置してください。このファイルは、プロジェクトに不慣れな関係者（同僚、スタッフ、採用担当者など）が、プロジェクトの概要、実行方法、関連情報の入手先を素早く理解できるようにするためのものです。

README.mdには少なくとも以下を含める必要があります：

- 最初の行は斜体で表記し、以下のように記述すること：*This project has been created as part of the 42 curriculum by <login1>[, <login2>[, <login3>[...]]]*.
- プロジェクトの目的と概要を明確に示す「説明」セクション。
- コンパイル、インストール、実行に関する関連情報を記載した「Instructions」セクション。
- 「リソース」セクションには、トピックに関連する古典的な参考文献（ドキュメント、記事、チュートリアルなど）をリストアップするとともに、AIがどのように使用されたか（どのタスクやプロジェクトのどの部分で）を具体的に記述します。

➡ プロジェクトに応じて追加セクションが必要となる場合があります（例：使用例、機能一覧、技術的选择など）。

必要な追加項目は以下に明示的に記載します。

- **プロジェクト説明**セクションでは、Dockerの使用方法和プロジェクトに含まれるソースについても説明する必要があります。主な設計上の選択事項を明示するとともに、以下の比較を記載すること：
 - 仮想マシン vs Docker
 - シークレット vs 環境変数
 - Dockerネットワーク vs ホストネットワーク
 - Dockerボリューム vs バインドマウント

第7章

検証の前提条件

既存の要件に加え、以下のドキュメントファイルをリポジトリのルートディレクトリに配置する必要があります。これらはMarkdown形式（.md）で記述されている必要があります。

- **USER_DOC.md** — ユーザードキュメントこのファイルでは、エンドユーザーまたは管理者が以下の操作を、明確かつ簡潔な言葉で説明する必要があります：
 - スタックが提供するサービスを理解する。
 - プロジェクトの起動と停止
 - ウェブサイトと管理パネルにアクセスする方法
 - 認証情報を検索・管理する方法
 - サービスが正しく動作していることを確認する。
- **DEV_DOC.md** — **開発者向けドキュメント** このファイルでは、開発者が以下の操作を実行する方法を記述する必要があります：
 - 環境を一から構築する（前提条件、設定ファイル、シークレット）。
 - MakefileとDocker Composeを使用してプロジェクトをビルドし起動する。
 - コンテナとボリュームを管理するための関連コマンドを使用する。
 - プロジェクトデータの保存場所と永続化方法を特定する。

第VIII章 ボーナスポー

ト

このプロジェクトでは、ボナスポーはシンプルに設計されています。

追加サービスごとにDockerfileを作成する必要があります。これにより、各サービスは独自のコンテナ内で実行され、必要に応じて専用のボリュームを持つことになります。

ボナスポーリスト：

- WordPressサイトのキャッシュを適切に管理するため、Redisキャッシュを設定する。
- WordPressウェブサイトのボリュームを指すFTPサーバーコンテナを設定してください。
- PHPを除く任意の言語（PHPは除外）でシンプルな静的ウェブサイトを作成します。例えば、作品紹介サイトや履歴書公開サイトなどです。
- Adminerを設定してください。
- 有用と思われる任意のサービスを設定してください。発表時にはその選択理由を説明する必要があります。



ボナスポー部分として、追加サービスの設定が可能です。この場合、必要に応じてポートを開放できます。



ボナスポー部分は、必須部分が完全に完了した場合にのみ評価されます。完全に完了とは、必須部分が完全に実施され、不具合なく機能している状態を指します。必須要件のすべてを満たしていない場合、ボナスポー部分は一切評価されません。

第9章

提出とピア評価

課題は通常通りGitリポジトリに提出してください。審査ではリポジトリ内の作業のみが評価対象となります。フォルダ名やファイル名が正しいか、必ず再確認してください。

評価中に、**プロジェクトの簡単な修正**を求められる場合があります。これは、動作の微調整、数行のコードの追加・修正、あるいは簡単に追加できる機能の追加などを含む可能性があります。

この手順は**全てのプロジェクトに適用されるわけではありませんが**、評価ガイドラインに記載されている場合は必ず対応できるように準備してください。

このステップは、プロジェクトの特定部分に対する実際の理解度を確認する目的です。修正は任意の開発環境（普段使用している環境など）で実施可能であり、評価に特定の時間枠が定められていない限り、数分以内に完了できる範囲であるべきです。

例えば、関数やスクリプトの小さな更新、表示の変更、新しい情報を保存するためのデータ構造の調整などを依頼される場合があります。

詳細（範囲、対象など）は**評価ガイドライン**で指定され、同じプロジェクトでも評価ごとに異なる場合があります。